

Cloud infrastructure

[Menu](#)[On this page](#)

Séance 5

Réseau de base : Service

Les pods Kubernetes sont mortels : ils naissent et lorsqu'ils meurent, ils ne ressuscitent pas.

Si vous utilisez un déploiement pour exécuter votre application, celui-ci peut créer et détruire dynamiquement des pods.

Chaque pod obtient sa propre adresse IP, mais dans un déploiement, l'ensemble de pods s'exécutant en un instant peut être différent de l'ensemble de pods exécutant cette application un instant plus tard.

Cela conduit à un problème : si un ensemble de pods (appelons-les « *backends* ») fournit des fonctionnalités à d'autres pods (appelons-les « *frontends* ») à l'intérieur de votre cluster, comment les pods *frontends* peuvent-ils trouver et suivre l'adresse IP à laquelle se connecter, afin que le frontend puisse utiliser la partie backend de la charge de travail?

C'est là où les **services** entrent en jeu.

Objectifs

- Comprendre les différents types de service
- Comprendre les sélecteurs
- *Les pods sont accessibles de l'extérieur via une IP de load balancer sur un port spécifique*

Remarque

À partir de cette séance, toute création de pod doit être réalisée via un *Deployment*.

Tâche 1

Vous déployez un web service dans un cluster Kubernetes. Vous voulez l'exposer en interne en utilisant un *Service*.

Un [Service](#) Kubernetes est une couche d'abstraction qui définit un ensemble de *Pod* et une règle pour y accéder. Il fournit un endpoint réseau stable ainsi qu'un service de répartition de charge vers les *Pod* sélectionnés.

De manière déclarative, créez un *Service* qui expose un *Pod* utilisant l'image [docker.io/patteantoine/5clo1r:tagname](https://patteantoine.io/5clo1r:tagname)

Exemple d'un service simple de type *ClusterIP*

yaml

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: ClusterIP
```

Exigences

Le *Service* doit être de type *ClusterIP*

Le *Service* doit exposer le port interne de l'application (TCP/5000) sur le premier port du range qui vous est attribué

Le *Label* à utiliser sur le *Pod* et le *selector* du *Service* doivent être *app.kubernetes.io/name* avec la valeur *my-app*

Questions

Quel est le contenu du fichier YAML pour créer le *Service* ainsi que le *Deployment* ?

Quelle commande `kubect1` faut-il utiliser pour réaliser un port-foward sur le *Service* ?

Tâche 2

De manière déclarative, créez un *Service* qui expose un *Pod* utilisant l'image [docker.io/patteantoine/5clo1r:tagname](https://patteantoine.github.io/5clo1r:tagname) permettant d'y accéder via une IP de type *Load Balancer*.

Exigences

Le *Service* doit être de type *LoadBalancer*

Le *Service* doit exposer le port interne de l'application (TCP/5000) sur le second port du range qui vous est attribué

Les *Label* à utiliser sur le *Pod* et le *selector* du *Service* doivent être *app.kubernetes.io/name* avec la valeur *my-app* ainsi que *app.kubernetes.io/service* avec la valeur *by-load-balancer*

Le ou les *Pod* de la tâche précédente ne doivent pas être sélectionnés par ce second *Service*

Questions

Quel est le contenu du fichier yaml pour créer le *Service* ainsi que le *Deployment* ?

Quelle commande `kubect1` faut-il utiliser pour connaître les différents endpoints ciblés par le *Service* ? Expliquez l'output de la commande.

Quelle est l'ip *Load Balancer* assignée aux services ? Comment la trouve-t-on ?

Quelle est la différence entre un service de type *ClusterIP* et un service de type *Load Balancer* ?

Tâche 3

Faites varier le nombre de *replicas* sur votre *Deployment* précédemment créé.

Question

Que pouvez-vous observer sur le *Service* ?

Mise à jour le: lundi 15 décembre 2025 à 17:16

Previous page
[Séance 4](#)

Next page
[Séance 6](#)